

KubeGraphRisk: A Weakly Supervised Heterogeneous Graph Neural Network for Failure-Rate Prediction in Kubernetes-Based Big-Data Platforms

A new method synthesized from Paper 5 and Paper 6

Full SCI-Style Research Manuscript Draft

Recommended Pair 2: Paper 5 + Paper 6

Author name(s) omitted for review

Affiliation and corresponding-author information to be added

Research integrity note: This manuscript proposes a new unified method. Numerical results attributed to the source studies are clearly identified as prior evidence; no experimental results for the proposed unified model are fabricated.

Abstract

Kubernetes is widely used to orchestrate microservices, streaming analytics, machine-learning pipelines, and big-data workloads, yet most platform-monitoring mechanisms remain reactive. Existing research provides two complementary solutions: GATAKU uses graph convolutional networks over Kubernetes network-flow data for accurate anomaly detection, while a separate Kubernetes resilience framework represents cluster resources as a dynamic heterogeneous graph in Neo4j and combines Prometheus telemetry with unsupervised and supervised learning for anomaly detection, prediction, and hierarchical localization. This paper proposes KubeGraphRisk, a new framework that unifies these strengths and extends them toward explicit multi-horizon failure-rate forecasting. The cluster is modeled as a native heterogeneous knowledge graph whose nodes correspond to Kubernetes resources and whose edges encode actual ownership, scheduling, storage, selection, and runtime communication relations. Prometheus and Thanos provide resource telemetry, Cilium and Hubble provide eBPF-based network flows, and Kubernetes Events supply failure-state transitions. An Isolation Forest generates high-confidence weak labels when verified incident annotations are scarce. A relation-aware heterogeneous graph neural network replaces artificial k-nearest-neighbor graph construction and independent resource classifiers, while a temporal encoder models degradation trajectories. Multi-task outputs estimate current anomaly probability, future failure probability, discrete failure hazard, and criticality-weighted platform failure rate. GNNExplainer-style subgraph extraction, attention weights, and Neo4j path traversal provide actionable explanations. The proposed evaluation includes operational faults, cybersecurity anomalies, temporal leakage controls, calibration analysis, early-warning lead time, and root-cause accuracy. KubeGraphRisk is designed as a practical bridge between graph-based anomaly detection and proactive reliability management for Kubernetes-based big-data platforms.

Keywords: Kubernetes; graph neural network; dynamic knowledge graph; failure-rate prediction; Cilium; Hubble; Prometheus; Neo4j; weak supervision; predictive maintenance

1. Introduction

Kubernetes has become a central execution layer for distributed applications, data platforms, artificial-intelligence services, and event-processing pipelines. A single cluster can contain thousands of Pods distributed across many Nodes, controlled by Deployments and StatefulSets, exposed through Services, attached to persistent volumes, and connected through continuously changing network flows. The health of such a platform depends on both resource state and topology. A memory leak in one Pod can pressure its Node, trigger eviction, reduce service capacity, increase latency, and indirectly overload downstream services.

Kubernetes includes self-healing mechanisms, but they are primarily reactive. A controller can recreate a failed Pod, and a scheduler can place a replacement elsewhere, yet such actions occur after a health condition or failure is observed. For high-value big-data workloads, it is preferable to estimate failure risk before a disruptive event. Predicting that a Node has a 75% probability of failure within one hour is operationally more useful than reporting that it is currently anomalous.

Paper 5, GATAKU, demonstrates that GCNs can successfully classify Kubernetes network anomalies when network packets are transformed into a graph. It uses Cilium for detailed network observability, feature engineering, k-nearest-neighbor graph construction, a three-layer GCN, and GNNExplainer. Paper 6 takes a different route: Kubernetes resources are represented as a dynamic heterogeneous Neo4j graph; Prometheus and Thanos provide monitoring; Isolation Forest, DBSCAN, and One-Class SVM detect anomalies; supervised classifiers predict labels; and hierarchical graph analysis localizes defective Pods and Nodes.

These solutions are complementary but incomplete for failure-rate prediction. GATAKU learns with a deep graph model but uses an artificial similarity graph and focuses mainly on current security anomaly classes. Paper 6 uses real Kubernetes topology and future anomaly prediction but relies on conventional machine-learning models rather than message passing over the graph. KubeGraphRisk combines actual Kubernetes semantics, network-flow GNNs, weak supervision, temporal modeling, and survival-style failure prediction in one system.

The main contributions are: a native dynamic heterogeneous Kubernetes knowledge graph augmented with runtime communication edges; fusion of Prometheus/Thanos and Cilium/Hubble telemetry; weak supervision based on high-confidence unsupervised labels; relation-aware GNN learning directly on the operational graph; multi-horizon component failure probability and hazard estimation; and graph-grounded root-cause and propagation explanations.

2. Foundations from the Two Source Solutions

2.1 GATAKU and network-flow graph learning

GATAKU builds a Kubernetes environment, collects inter-container network data using Cilium, simulates attack scenarios, engineers features, and constructs a graph using k-nearest neighbors. A three-layer GCN then performs anomaly classification. In the reported controlled dataset, the GCN achieved mean precision, recall, accuracy, and F1 values around 0.999 for both binary and multiclass experiments. GNNExplainer was used to identify influential graph structures and features.

The main strength is clear: graph neural networks can exploit relationships in Kubernetes traffic data and produce highly accurate predictions under the experimental conditions. The main limitation for operational reliability forecasting is that feature similarity is not the same as real Kubernetes dependency. Two records can be numerically similar without being operationally connected, while two dependent Pods may have very different local metrics.

2.2 Dynamic heterogeneous Kubernetes graph and two-stage prediction

Paper 6 models the cluster as a dynamic graph containing Kubernetes resources and their relationships. Prometheus provides metrics, Thanos supports long-term monitoring, and Neo4j stores graph structure. The proposed learning process is separated into unsupervised anomaly detection and supervised prediction. Isolation Forest achieved the strongest reported silhouette score of 0.7941 among the compared unsupervised models, and the Decision Tree achieved a best F1 score of 0.8864 among the supervised classifiers. The graph is then used to trace affected resources hierarchically.

The main strength is operational semantics: the graph knows that a Pod is managed by a ReplicaSet, belongs to a Namespace, and runs on a Node. The limitation is that the classifiers do not directly perform graph message passing, so structural context must be encoded manually or used mainly after prediction.

2.3 Synthesis rationale

KubeGraphRisk preserves the real graph from Paper 6 and the deep graph-learning principle from Paper 5. Network flows become actual COMMUNICATES_WITH edges rather than observations connected by kNN similarity. Unsupervised learning remains valuable, but its labels become weak supervision for a deep heterogeneous GNN rather than final truth. The new target is future operational failure rather than only current anomaly class.

Table 1. Complementarity of Papers 5 and 6

Dimension	Paper 5: GATAKU	Paper 6: Kubernetes resilience	KubeGraphRisk
Primary graph	kNN similarity graph over flow observations	Native Kubernetes heterogeneous graph	Native heterogeneous graph + runtime flow edges
Learning	Three-layer GCN	Isolation Forest + classic classifiers	Weakly supervised heterogeneous GNN + temporal model
Primary target	Current anomaly / attack class	Anomaly detection and prediction	Current anomaly + multi-horizon failure risk
Explainability	GNNExplainer	Graph hierarchy and visualization	GNNExplainer + attention + graph-path RCA
Telemetry	Cilium network data	Prometheus/Thanos resource data	Both modalities plus events

3. Problem Definition

Let G_t represent the Kubernetes cluster at time t . Vertices are resources such as Nodes, Pods, Containers, Deployments, ReplicaSets, Services, StatefulSets, PersistentVolumes, PersistentVolumeClaims, Namespaces, and selected application components. Edges represent native relationships such as RUNS_ON, MANAGED_BY, OWNS, SELECTS, EXPOSES, MOUNTS, and COMMUNICATES_WITH.

For each component v , the model observes a historical window of graph states and predicts whether the component is currently anomalous and whether it will fail within one or more future horizons. Failure is defined operationally rather than statistically. Examples include NodeNotReady, unrecoverable container termination, sustained service unavailability, repeated health-check failure followed by removal, OOMKilled with workload disruption, storage loss, or severe SLO violation.

$$G_t = (V_t, E_t, X_t, R) \quad (1)$$

$$p_hat_v(\Delta)^t = P(T_v \leq t + \Delta \mid T_v > t, G_{[t-L+1:t]}) \quad (2)$$

4. KubeGraphRisk System Architecture

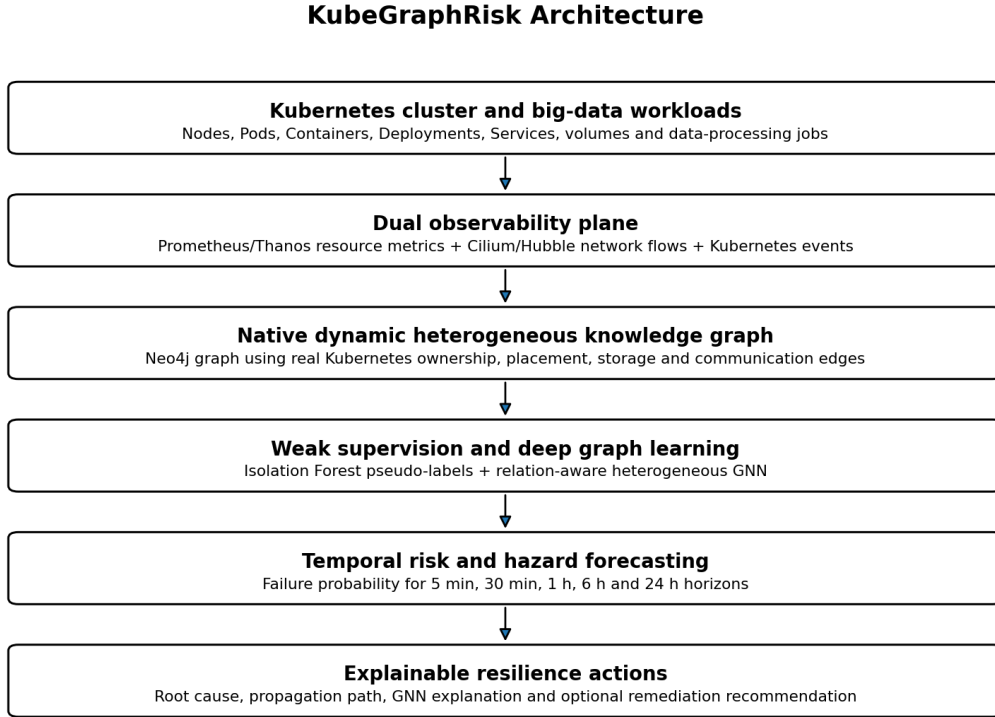


Figure 1. Proposed KubeGraphRisk architecture.

The architecture contains six stages. The observability plane collects infrastructure metrics, network flows, and events. The graph builder updates Neo4j or another graph store. The weak-supervision engine generates provisional labels from resource-type-specific unsupervised detectors. The deep graph encoder learns cross-resource context. The temporal risk module predicts future failure probability and hazard. The explanation layer returns root causes, influential metrics, and candidate propagation paths.

This modular design allows online inference to run at short intervals while slower pseudo-label updates and retraining occur less frequently. The production graph remains useful independently of the learning model because operators can query it directly for dependency and ownership information.

5. Telemetry and Native Knowledge Graph

5.1 Resource telemetry

Prometheus scrapes infrastructure and application metrics. Thanos can provide long-term retention, cross-cluster queries, and historical data for model training. Features include CPU usage, CPU throttling, memory working set, memory limits, filesystem utilization, disk I/O latency, network throughput, error rate, restart count, readiness status, liveness status, replica availability, and application latency.

Each resource type uses a different feature schema. Nodes are compared with Nodes, Pods with Pods, and Containers with Containers. Type-specific normalization prevents scale differences from dominating the model.

5.2 Network-flow telemetry

Cilium and Hubble provide eBPF-based network visibility. Each flow can include source and destination identities, namespaces, Pods, Services, protocol, ports, packet counts, byte counts, verdicts, latency-related measures, and dropped or denied traffic. Rather than constructing a kNN graph over flow records, KubeGraphRisk aggregates flows over a time window and adds or updates COMMUNICATES_WITH edges between real entities.

Edge attributes can include request volume, byte volume, error ratio, port diversity, retransmission indicators, and change from baseline. This preserves the valuable feature engineering of GATAKU while anchoring the information to true topology.

5.3 Kubernetes events and failure labels

Kubernetes Events provide explicit state transitions such as FailedScheduling, BackOff, Unhealthy, Evicted, NodeNotReady, and volume errors. Container termination reasons such as OOMKilled and application exit codes provide additional labels. To prevent circularity, only events that occur before the prediction cutoff are used as model inputs; future events define targets.

Table 2. Representative Kubernetes graph entities

Graph node type	Example edges	Example features
Node	hosts Pod; member of Cluster	CPU, memory pressure, disk pressure, network, readiness
Pod	managed by ReplicaSet; runs on Node; communicates with Pod	CPU, memory, restarts, probe failures, latency
Service	selects Pod; communicates with Service	request volume, errors, dependency fan-out
Volume	mounted by Pod; attached to Node	capacity, latency, errors, attachment state
Deployment/StatefulSet	owns ReplicaSet/Pod	desired vs available replicas, rollout state

6. Weak Supervision

Verified failure labels are scarce in production, and historical incident tickets often identify only the affected service rather than every contributing component. KubeGraphRisk therefore uses weak supervision. For each resource type, an Isolation Forest is trained on an assumed-normal or mixed historical dataset. High anomaly scores above θ_{high} are treated as provisional positive labels, while scores below θ_{low} become provisional normal labels. Ambiguous samples are excluded from the initial supervised loss.

Verified incident labels, controlled fault-injection labels, and deterministic Kubernetes failure events override pseudo-labels. The resulting dataset contains a mixture of strong and weak labels, each with a confidence weight. The deep model therefore learns from more data without treating noisy pseudo-labels as perfect truth.

$$y_{\text{tilde}}^v = 1 \text{ if } s_v \geq \theta_{\text{high}}; 0 \text{ if } s_v \leq \theta_{\text{low}}; \text{unknown otherwise} \quad (3)$$

$$L_{\text{weak}} = \sum_i c_i * \text{BCE}(y_i, \hat{p}_i) \quad (4)$$

7. Heterogeneous Graph Neural Network

A homogeneous GCN applies one transformation to all neighbors, but Kubernetes relations have different semantics. RUNS_ON represents infrastructure placement, MANAGED_BY represents control hierarchy, MOUNTS represents storage dependency, and COMMUNICATES_WITH represents runtime interaction. KubeGraphRisk therefore uses relation-specific transformations and attention coefficients.

Each node type has an encoder that projects raw features into a shared latent dimension. During message passing, the model aggregates information separately by relation type and learns attention weights. This enables, for example, a Node to receive evidence from resident Pods while a Service receives evidence from selected Pods and communication partners.

$$h_v(0) = \phi_{\tau}(v)(x_v) \quad (5)$$

$$m_{(u \rightarrow v)}^r = W_r h_u \quad (6)$$

$$\alpha_{(u,v)}^r = \text{softmax}_v(\text{LeakyReLU}(a_r^T [W_r h_u \parallel W_r h_v \parallel e_{uv}])) \quad (7)$$

$$h_v(l+1) = \sigma(W_0 h_v^l + \sum_r \sum_{u \in N_r(v)} \alpha_{(u,v)}^r W_r h_u^l) \quad (8)$$

8. Temporal Failure Forecasting

A single snapshot cannot distinguish a brief spike from progressive degradation. For every component, graph embeddings from the last L snapshots are fed into a causal GRU, LSTM, or temporal Transformer. The temporal state summarizes trend, volatility, recovery, and interactions with neighboring components.

The model produces separate heads for current anomaly, future failure probability at multiple horizons, and discrete hazard. Multi-horizon outputs may include 5 minutes, 30 minutes, 1 hour, 6 hours, and 24 hours. The horizons should be selected according to operational remediation time.

$$z_v^t = \text{TemporalEncoder}(h_v(t-L+1), \dots, h_v^t) \quad (9)$$

$$\hat{p}_{\text{hat}}(v, \Delta_j)^t = \text{sigmoid}(W_{\Delta_j} z_v^t + b_{\Delta_j}) \quad (10)$$

$$P(T_v \leq K) = 1 - \text{product}_{\{k=1..K\}}(1 - \hat{q}_{\text{hat}}(v, k)) \quad (11)$$

$$\lambda_{\text{hat}}(v, \Delta) = -\ln(1 - \hat{p}_{\text{hat}}(v, \Delta)) / \Delta \quad (12)$$

9. Explainability and Root-Cause Analysis

The explanation module returns three layers of evidence. GNNExplainer or a related subgraph explainer identifies the compact set of nodes, edges, and features that most influence a prediction. Learned relation attention indicates which neighbors and relation types were important. Neo4j path traversal converts these entities into an operational explanation.

A high-risk prediction might therefore be expressed as: Node-3 has a predicted 0.78 probability of failure within 30 minutes. Pod analytics-api-17 is the strongest contributing child resource because of rising memory usage and restart frequency. It runs on Node-3 and communicates with Service-A, where latency has also increased. The candidate propagation path is Pod-17 -> Node-3 -> Service-A.

Such explanations are intended to help operators prioritize investigation. They are not treated as definitive causal proof.

10. Training Objective and Class Imbalance

The complete loss combines anomaly classification, future failure classification, survival likelihood, confidence-weighted weak labels, and optional graph smoothness. Focal loss is suitable for rare failures, but calibration must be checked because aggressive class weighting can distort probabilities.

Sampling should preserve incident integrity. Windows from the same failure episode must not be split across training and test sets. Hard negative mining can focus learning on anomalous-but-nonfailing periods, which are particularly important for distinguishing transient deviations from genuine degradation.

$$L = \beta_a L_{\text{anomaly}} + \beta_f L_{\text{failure}} + \beta_h L_{\text{hazard}} + \beta_w L_{\text{weak}} + \beta_s L_{\text{graph}} \quad (13)$$

11. Experimental Design

The proposed testbed should contain at least four Kubernetes worker nodes, multiple Namespaces, a service mesh or observable communication layer, persistent storage, and representative big-data workloads. Suitable applications include a streaming pipeline, an API tier, a database, a message broker, and batch-processing components. Prometheus, Thanos, Cilium, Hubble, Neo4j, and a deep-learning framework such as PyTorch Geometric form the observability and modeling stack.

The experiment should inject both operational and cybersecurity anomalies. Operational scenarios include CPU saturation, memory leak, memory exhaustion, OOMKilled, CrashLoopBackOff, probe failure, Node memory pressure, disk pressure, storage latency, network delay, packet loss, bandwidth throttling, broken service connectivity, and replica loss. Security scenarios can include denial-of-service, port scanning, and vulnerability scanning to preserve comparability with GATAKU.

Each scenario should include multiple severity levels and random start times. The evaluation records fault onset, first model alert, failure time, recovery time, affected graph entities, and true root cause. This supports detection, forecasting, and RCA metrics simultaneously.

Table 3. Evaluation criteria

Metric	Reason
AUPRC	Preferred discrimination metric for rare failures
F1 and recall	Operational balance and missed-failure rate
Brier score / ECE	Calibration of predicted probabilities
Mean warning lead time	Time available for preventive action
C-index / time-dependent AUC	Time-to-event forecasting quality
Top-k RCA accuracy	Whether the true root cause is ranked highly
Inference latency	Whether online prediction is feasible

12. Baselines and Ablations

The baseline set should include Isolation Forest, One-Class SVM, Decision Tree, Random Forest, SVM, an LSTM-only temporal model, a GATAKU-style kNN-GCN, and a graph-free supervised model using aggregated topology features. The full KubeGraphRisk model should be ablated by removing network edges, native topology, weak supervision, temporal encoding, relation typing, and the survival head.

The most important experiment compares three graph definitions under the same deep architecture: no graph, kNN similarity graph, and native Kubernetes graph augmented with runtime communication. This directly tests the central hypothesis that operational semantics improve failure prediction and root-cause localization.

Table 4. Planned ablation studies

Ablation	Removed component	Expected insight
B1	Native topology, use kNN only	Value of semantic Kubernetes relations
B2	Network-flow edges	Value of Cilium/Hubble runtime context
B3	Weak supervision	Benefit under limited verified labels
B4	Temporal encoder	Value of degradation history
B5	Relation-aware attention	Need for heterogeneous modeling
B6	Survival/failure head	Difference between anomaly detection and failure forecasting

13. Evidence from Papers 5 and 6

The reported values in this section belong to the source studies and are not experimental results of KubeGraphRisk. In Paper 5, the GCN achieved mean precision, recall, accuracy, and F1 values close to 0.999 in the study's controlled single-class and multiclass network-anomaly experiments. These results show that GNNs can be highly effective on the constructed Kubernetes traffic dataset, although production generalization and operational failure forecasting remain separate questions.

In Paper 6, Isolation Forest obtained a reported silhouette score of 0.7941, and the Decision Tree achieved a best F1 score of 0.8864. The graph-based architecture was also used to trace anomalous resources through Kubernetes relationships. These results support the feasibility of dynamic topology modeling and two-stage learning.

KubeGraphRisk uses these studies as complementary evidence: deep graph learning is effective, and native resource graphs are operationally meaningful. The research question is whether a unified native graph with weak supervision and temporal failure objectives can produce earlier and better-calibrated predictions than either approach alone.

14. Discussion

The central architectural choice is to treat Kubernetes as the graph it already is. A Pod is placed on a Node, selected by a Service, owned by a controller, connected to storage, and communicating with other workloads. Converting packet observations into a kNN graph can be useful, but it discards these semantics. KubeGraphRisk instead retains actual Kubernetes relations and uses similarity only as an optional auxiliary edge type.

Weak supervision is also important. A purely supervised deep model may be unrealistic because organizations rarely maintain perfect labels for every failure precursor. However, pseudo-labels must be treated cautiously. High-confidence thresholds, confidence weights, and verified-label overrides reduce the risk of training on noisy anomaly assignments.

The proposed failure-rate output creates a natural bridge to reliability engineering. Operators can compare component risk, aggregate it by service or namespace, and define action policies based on probability, criticality, and propagation risk. For example, a high-risk low-priority batch Pod may be restarted automatically, whereas a moderate-risk control-plane Node may trigger immediate human review.

15. Limitations and Threats to Validity

KubeGraphRisk is a proposed unified framework and has not yet produced its own measured results. The source studies use controlled datasets that may not represent all production conditions. Security anomalies and operational failures have different dynamics, and one model may not optimally cover both. Graph size and topology churn can create inference overhead. Explanations derived from GNN methods may be unstable across retraining runs, and attention should not be interpreted as causal proof.

The failure label definition is critical. A container restart is not always a business-impacting failure, while a sustained latency violation may be severe even when every Kubernetes resource remains Ready. Evaluation should therefore report component-level and service-level targets separately.

16. Conclusion

This paper proposed KubeGraphRisk, a weakly supervised heterogeneous graph neural framework for multi-horizon failure-rate prediction in Kubernetes-based big-data platforms. The method combines the deep graph-learning and network-observability strengths of GATAKU with the native dynamic graph, Prometheus/Thanos telemetry, two-stage learning, and hierarchical localization strengths of the Kubernetes resilience framework.

The resulting system uses actual Kubernetes relationships, runtime communication edges, weak labels, relation-aware message passing, temporal degradation modeling, calibrated failure probabilities, survival hazard, and graph-grounded explanations. The next step is to implement the full architecture and evaluate it under controlled and natural incidents using temporal splits, calibration metrics, early-warning lead time, and root-cause accuracy.

References

- [1] M. Osswald, T. Schonenberger, G. Cantali, W. Soussi, and G. Gur, "Anomaly Detection in Microservices Architecture Using Graph Neural Networks," 33rd Euromicro International Conference on Parallel, Distributed, and Network-Based Processing, pp. 560-567, 2025, doi: 10.1109/PDP66500.2025.00085.
- [2] V. Anemogiannis, B. Andreou, K. Myrtollari, K. Panagidi, and S. Hadjiefthymiades, "Enhancing Kubernetes Resilience through Anomaly Detection and Prediction," arXiv:2503.14114, 2025.
- [3] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," International Conference on Learning Representations, 2017.
- [4] P. Velickovic et al., "Graph Attention Networks," International Conference on Learning Representations, 2018.
- [5] Z. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, "GNNExplainer: Generating Explanations for Graph Neural Networks," Advances in Neural Information Processing Systems, vol. 32, 2019.
- [6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," IEEE International Conference on Data Mining, pp. 413-422, 2008.
- [7] B. Scholkopf, R. C. Williamson, A. Smola, J. Shawe-Taylor, and J. Platt, "Support Vector Method for Novelty Detection," Advances in Neural Information Processing Systems, vol. 12, 1999.
- [8] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," KDD, pp. 226-231, 1996.
- [9] J. Kosinska and M. Tobiasz, "Detection of Cluster Anomalies with Machine Learning Techniques," IEEE Access, vol. 10, pp. 110742-110753, 2022.
- [10] C.-W. Tien et al., "Kubanomaly: Anomaly Detection for the Docker Orchestration Platform with Neural Network Approaches," Engineering Reports, vol. 1, no. 5, e12080, 2019.
- [11] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
- [12] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," Advances in Neural Information Processing Systems, vol. 30, 2017.
- [13] Y. Dang, Q. Lin, and P. Huang, "AIOps: Real-World Challenges and Research Innovations," IEEE/ACM ICSE Companion, pp. 4-5, 2019.
- [14] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," ACM CCS, pp. 1285-1298, 2017.
- [15] D. P. Kingma and J. Ba, "Adam: A Method for Stochastic Optimization," International Conference on Learning Representations, 2015.